

CI2024_project-work

Symbolic Regression Using Genetic Programming

Introduction

This report documents the implementation and reasoning behind the design of a **Genetic Programming** (GP) approach to **Symbolic Regression**. The goal of this project is to evolve mathematical expressions that approximate a target function using genetic programming techniques such as selection, crossover, and mutation.

Problem Definition

The problem consists of evolving symbolic expressions that approximate an unknown function given training data (X, y) . The dataset is stored in `.npz` files, where:

- `x`: Input features.
- `y`: Target outputs.

We initialize the dataset as follows:

```
problem = np.load('problem_0.npz')
x = problem['x']
y = problem['y']
```

Representation of Individuals

Each individual in the population is represented as a **tree**, where:

- **Operators** (`+`, `-`, `*`, `/`) and **Functions** (`sin`, `cos`, `exp`, `log`) are internal nodes.
- **Variables** (`x0`, `x1`, ...) and **Constants** (`pi`, `e`, `random numbers`) are leaf nodes.

Node Class

```
class Node:
    def __init__(self, type: str, value: str, parent=None, left=None, right=None):
        self.type = type # 'var', 'const', 'math_const', 'unary_op', 'binary_op'
        self.value = value
        self.parent = parent
        self.left = left
        self.right = right

    def copy(self):
        new_node = Node(self.type, self.value)
        if self.left:
            new_node.left = self.left.copy()
```

```

        new_node.left.parent = new_node
    if self.right:
        new_node.right = self.right.copy()
        new_node.right.parent = new_node
    return new_node

```

Generating Random Trees

To create a diverse initial population, we use a **random tree generator** that supports both the **"grow"** and **"full"** methods. The weights for selecting node types are chosen to balance diversity and complexity:

- **Grow method** allows for early stopping and results in variable-depth trees.
- **Full method** ensures the tree is completely filled up to `max_depth`.
- **Node type probabilities** (0.05, 0.2375, 0.2375, 0.2375, 0.2375) are set to prioritize balanced exploration between constants, variables, and operators.
- The probability of choosing constants is slightly lower than variables to encourage function discovery over simple numerical fitting.

```

def generate_random_tree(num_vars, mode='full', max_depth=5, max_const=10,
depth=0):
    if depth == max_depth:
        node_type = random.choices(['math_const', 'const', 'var'], weights=[0.1,
0.4, 0.5])[0]
        if node_type == 'const':
            value = str(random.randint(0, max_const))
        elif node_type == 'math_const':
            value = random.choice(['pi', 'e'])
        else:
            value = 'x' + str(random.randint(0, num_vars - 1))
        return Node(node_type, value)

    if mode == 'full':
        node_type = random.choice(['unary_op', 'binary_op'])
    elif mode == 'grow':
        node_type = random.choices(['const', 'math_const', 'var', 'unary_op',
'binary_op'],
                                weights=[0.05, 0.2375, 0.2375, 0.2375, 0.2375])
[0]

    if node_type == 'unary_op':
        left_child = generate_random_tree(num_vars, mode, max_depth, max_const,
depth + 1)
        return Node('unary_op', random.choice(['sin', 'cos', 'exp', 'log']),
left=left_child)
    elif node_type == 'binary_op':
        left_child = generate_random_tree(num_vars, mode, max_depth, max_const,
depth + 1)
        right_child = generate_random_tree(num_vars, mode, max_depth, max_const,
depth + 1)

```

```
        return Node('binary_op', random.choice(['+', '-', '*', '/']),
                    left=left_child, right=right_child)
```

Fitness Function

The fitness function evaluates how well a tree approximates the target function using **Mean Squared Error (MSE)** with a complexity penalty. The penalty coefficient (`lambda_penalty = 0.01`) prevents overfitting by discouraging excessive tree depth. This value was chosen based on empirical testing, as higher values excessively penalized deeper, potentially useful trees.

```
def fitness(tree, x, y, lambda_penalty=0.01):
    eval_y = evaluate_tree(tree, x)
    if np.any(np.isinf(eval_y)):
        return float('inf')
    mse = np.mean((eval_y - y) ** 2)
    tree_depth = get_tree_depth(tree)
    return mse + lambda_penalty * tree_depth
```

Selection: Tournament Selection

Tournament selection ensures that fitter individuals have a higher probability of being selected while maintaining genetic diversity. The tournament size is set to `2`, ensuring a balance between selection pressure and diversity preservation. A larger tournament size would increase selection pressure, potentially leading to premature convergence.

```
def tournament_selection(population, fitness_dict, tournament_size=2):
    tournament = random.sample(population, tournament_size)
    return min(tournament, key=lambda ind: fitness_dict[ind])
```

Crossover and Mutation

Crossover: Subtree Exchange

Subtree exchange allows for recombination of genetic material between two individuals. The randomness in selecting crossover points ensures diversity in the population while preserving functional substructures.

```
def crossover_trees(parent1, parent2):
    child1, child2 = parent1.copy(), parent2.copy()
    node1, node2 = random.choice(collect_nodes(child1)),
    random.choice(collect_nodes(child2))
    node1.parent, node2.parent = node2.parent, node1.parent
    return child1, child2
```

Mutations

Mutation introduces variation into the population. The mutation probability is set to **0.1** to maintain a balance between exploration and exploitation.

1. **Point Mutation:** Change a node value.
2. **Permutation Mutation:** Swap child nodes.
3. **Hoist Mutation:** Replace tree with a subtree.
4. **Collapse Mutation:** Replace subtree with a leaf.

```
def point_mutation(individual, num_vars, max_const=100):  
    nodes = collect_nodes(individual, take_root=True)  
    mutation_node = random.choice(nodes)  
    if mutation_node.type in ['math_const', 'const', 'var']:  
        new_type = random.choice(['math_const', 'const', 'var'])  
        if new_type == 'const':  
            mutation_node.value = str(random.randint(0, max_const))  
    return individual
```

This ensures convergence to optimal symbolic expressions while balancing exploration and exploitation.